

Phase 3 — Onboarding Wizard Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (– []) syntax for tracking.

Goal: Build a multi-step onboarding wizard (Welcome → Pick Providers → Configure → Summary) replacing the current flat single-screen onboarding.

Architecture: New :feature:onboarding module with Orbit MVI ViewModel + AnimatedContent step transitions. Four step composables. One ViewModel orchestrates step state, credential testing, and completion. Plugs into MainActivity via the existing showOnboarding flag.

Tech Stack: Jetpack Compose (AnimatedContent), Orbit MVI, Dagger Hilt, existing LavaTrackerSdk, ProviderCredentialManager, AuthService

Task 1: Create feature:onboarding module skeleton

Files:

- Create: feature/onboarding/build.gradle.kts
- Create: feature/onboarding/src/main/AndroidManifest.xml
- Modify: settings.gradle.kts



Step 1: Create build.gradle.kts

```
plugins {  
    id("lava.android.feature")  
    id("lava.android.hilt")  
}  
  
android {  
    namespace = "lava.onboarding"  
}  
  
dependencies {  
    implementation(project(":core:tracker:client"))  
    implementation(project(":core:auth:api"))  
    implementation(project(":core:credentials"))  
    implementation(project(":core:models"))  
    implementation(project(":core:domain"))  
}
```

```

implementation(project(":core:preferences"))
implementation(project(":core:designsystem"))
implementation(project(":core:ui"))
implementation(project(":feature:login"))

implementation(libs.kotlinx.coroutines.core)
implementation(libs.kotlinx.coroutines.android)

testImplementation(libs.junit4)
testImplementation(libs.kotlinx.coroutines.test)
testImplementation(libs.mockk)
}

```



Step 2: Create AndroidManifest.xml

```

<?xml version="1.0" encoding="utf-8"?>
<manifest />

```



Step 3: Register in settings.gradle.kts

Find the feature/* includes block. Add after the last feature module:

```
include(":feature:onboarding")
```



Step 4: Verify module resolves

Run: `./gradlew :feature:onboarding:tasks --group=help 2>&1 | head -5`
Expected: No resolution errors



Step 5: Commit

```

git add feature/onboarding/ settings.gradle.kts
git commit -m "feat(onboarding): create module skeleton

```

```

Bluff-Audit: N/A (module skeleton, no test code)
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

```

Task 2: Define state, action, side effect models

Files:

- Create: `feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingState.kt`
- Create: `feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingAction.kt`

- Create: feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingSideEffect.kt



Step 1: Create OnboardingState.kt

```
package lava.onboarding

import lava.tracker.api.TrackerDescriptor

enum class OnboardingStep { Welcome, Providers, Configure,
    Summary }

data class ProviderOnboardingItem(
    val descriptor: TrackerDescriptor,
    val selected: Boolean = true,
)

data class ProviderConfigState(
    val providerId: String,
    val username: String = "",
    val password: String = "",
    val useAnonymous: Boolean = false,
    val configured: Boolean = false,
    val tested: Boolean = false,
    val error: String? = null,
)

data class OnboardingState(
    val step: OnboardingStep = OnboardingStep.Welcome,
    val providers: List<ProviderOnboardingItem> = emptyList(),
    val configs: Map<String, ProviderConfigState> = emptyMap(),
    val currentProviderIndex: Int = 0,
    val connectionTestRunning: Boolean = false,
)
```



Step 2: Create OnboardingAction.kt

```
package lava.onboarding

sealed interface OnboardingAction {
    data object NextStep : OnboardingAction
    data object BackStep : OnboardingAction
    data class ToggleProvider(val providerId: String) :
        OnboardingAction
    data class UsernameChanged(val value: String) :
        OnboardingAction
    data class PasswordChanged(val value: String) :
        OnboardingAction
    data class ToggleAnonymous(val enabled: Boolean) :
        OnboardingAction
    data object TestAndContinue : OnboardingAction
}
```

```
    data object Finish : OnboardingAction
}
```



Step 3: Create OnboardingSideEffect.kt

```
package lava.onboarding
```

```
sealed interface OnboardingSideEffect {
    data object Finish : OnboardingSideEffect
}
```



Step 4: Run spotless + compile

```
Run: ./
gradlew :feature:onboarding:spotlessApply :feature:onboarding:compileDebugKotlin
```



Step 5: Commit

```
git add feature/onboarding/src/main/kotlin/lava/onboarding/
      OnboardingState.kt feature/onboarding/src/main/kotlin/
      lava/onboarding/OnboardingAction.kt feature/onboarding/
      src/main/kotlin/lava/onboarding/OnboardingSideEffect.kt
git commit -m "feat(onboarding): state, action, side effect models"
```

```
Bluff-Audit: N/A (model definitions, no test code)
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"
```

Task 3: Create OnboardingViewModel

Files:

- Create: feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingViewModel.kt



Step 1: Create OnboardingViewModel.kt

```
package lava.onboarding
```

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelScope
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.launch
import lava.auth.api.AuthService
import lava.credentials.ProviderCredentialManager
import lava.logger.api.LoggerFactory
import lava.tracker.client.LavaTrackerSdk
```

```

import lava.tracker.api.TrackerDescriptor
import org.orbitmvi.orbit.Container
import org.orbitmvi.orbit.ContainerHost
import org.orbitmvi.orbit.syntax.simple.intent
import org.orbitmvi.orbit.syntax.simple.postSideEffect
import org.orbitmvi.orbit.syntax.simple.reduce
import org.orbitmvi.orbit.viewmodel.container
import javax.inject.Inject

@HiltViewModel
class OnboardingViewModel @Inject constructor(
    private val sdk: LavaTrackerSdk,
    private val credentialManager: ProviderCredentialManager,
    private val authService: AuthService,
    loggerFactory: LoggerFactory,
) : ViewModel(), ContainerHost<OnboardingState,
    OnboardingSideEffect> {
    private val logger = loggerFactory.get("OnboardingViewModel")

    override val container: Container<OnboardingState,
        OnboardingSideEffect> = container(
        initialState = OnboardingState(),
        onCreate = { loadProviders() },
    )

    fun perform(action: OnboardingAction) {
        logger.d { "Perform $action" }
        when (action) {
            is OnboardingAction.NextStep -> onNextStep()
            is OnboardingAction.BackStep -> onBackStep()
            is OnboardingAction.ToggleProvider ->
onToggleProvider(action.providerId)
            is OnboardingAction.UsernameChanged ->
onUsernameChanged(action.value)
            is OnboardingAction.PasswordChanged ->
onPasswordChanged(action.value)
            is OnboardingAction.ToggleAnonymous ->
onToggleAnonymous(action.enabled)
            is OnboardingAction.TestAndContinue ->
onTestAndContinue()
            is OnboardingAction.Finish -> onFinish()
        }
    }

    private fun loadProviders() = intent {
        val descriptors = sdk.listAvailableTrackers().filter {
            it.verified && it.apiSupported
        }
        val items = descriptors.map { ProviderOnboardingItem(it) }
        val configs = items.associate {
            it.descriptor.trackerId to
            ProviderConfigState(it.descriptor.trackerId)
        }
    }

```

```

        reduce { state.copy(providers = items, configs =
            configs) }
    }

    private fun onNextStep() = intent {
        when (state.step) {
            OnboardingStep.Welcome -> reduce { it.copy(step =
                OnboardingStep.Providers) }
            OnboardingStep.Providers -> {
                val selected = state.providers.filter {
                    it.selected }
                if (selected.isEmpty()) return@intent
                reduce {
                    it.copy(step = OnboardingStep.Configure,
                        currentProviderIndex = 0)
                }
            }
            OnboardingStep.Configure -> advanceToNextProvider()
            OnboardingStep.Summary -> { /* ignore */ }
        }
    }

    private fun onBackStep() = intent {
        when (state.step) {
            OnboardingStep.Welcome ->
                postSideEffect(OnboardingSideEffect.Finish)
            OnboardingStep.Providers -> reduce { it.copy(step =
                OnboardingStep.Welcome) }
            OnboardingStep.Configure -> {
                if (state.currentProviderIndex == 0) {
                    reduce { it.copy(step =
                        OnboardingStep.Providers) }
                } else {
                    reduce { it.copy(currentProviderIndex =
                        state.currentProviderIndex - 1) }
                }
            }
            OnboardingStep.Summary -> { /* ignore */ }
        }
    }

    private fun onToggleProvider(providerId: String) = intent {
        reduce {
            it.copy(
                providers = it.providers.map { p ->
                    if (p.descriptor.trackerId == providerId)
                        p.copy(selected = !p.selected) else p
                },
            )
        }
    }

    private fun onUsernameChanged(value: String) =
        updateCurrentConfig { it.copy(username = value) }

```

```

private fun onPasswordChanged(value: String) =
    updateCurrentConfig { it.copy(password = value) }
private fun onToggleAnonymous(enabled: Boolean) =
    updateCurrentConfig { it.copy(useAnonymous = enabled) }

private fun updateCurrentConfig(transform:
    (ProviderConfigState) -> ProviderConfigState) = intent {
    val currentId = currentProviderId()
    if (currentId == null) return@intent
    reduce {
        it.copy(configs = it.configs + (currentId to
            transform(it.configs[currentId]!!)))
    }
}

private fun onTestAndContinue() = intent {
    val currentId = currentProviderId() ?: return@intent
    val config = state.configs[currentId] ?: return@intent
    val provider = state.providers.find {
        it.descriptor.trackerId == currentId }?.descriptor ?:
        return@intent

    reduce { it.copy(connectionTestRunning = true) }

    viewModelScope.launch {
        try {
            if (provider.authType ==
                lava.tracker.api.AuthType.NONE || config.useAnonymous) {
                sdk.switchTracker(currentId)
                val result = sdk.checkAuth(currentId)
                if (!result) {
                    reduce {
                        it.copy(
                            connectionTestRunning = false,
                            configs = it.configs + (currentId
                                to config.copy(error = "Connection failed")),
                        )
                    }
                    return@launch
                }
            } else {
                val loginResult = sdk.login(
                    currentId,
                    lava.tracker.api.model.LoginRequest(
                        username = config.username,
                        password = config.password,
                    ),
                )
                if (!loginResult.success) {
                    reduce {
                        it.copy(
                            connectionTestRunning = false,
                            configs = it.configs + (currentId
                                to config.copy(error = "Invalid credentials")),
                        )
                    }
                }
            }
        }
    }
}

```

```

        )
    }
    return@launch
}
credentialManager.setPassword(currentId,
config.username, config.password)
}

reduce {
    it.copy(
        connectionTestRunning = false,
        configs = it.configs + (currentId to
config.copy(configured = true, tested = true)),
    )
}
advanceToNextProvider()
} catch (e: Exception) {
    reduce {
        it.copy(
            connectionTestRunning = false,
            configs = it.configs + (currentId to
config.copy(error = e.message ?: "Connection error")),
        )
    }
}
}
}

private fun advanceToNextProvider() = intent {
    val selected = state.providers.filter { it.selected }
    val nextIndex = state.currentProviderIndex + 1
    if (nextIndex < selected.size) {
        reduce { it.copy(currentProviderIndex = nextIndex) }
    } else {
        reduce { it.copy(step = OnboardingStep.Summary) }
    }
}

private fun onFinish() = intent {
    val configured = state.configs.filter {
        it.value.configured
    }
    for ((providerId, config) in configured) {
        val desc = state.providers.find {
            it.descriptor.trackerId == providerId
        }?.descriptor ?:
        continue
        authService.signalAuthorized(
            name = if (config.useAnonymous) "Anonymous" else
            config.username,
            avatarUrl = null,
        )
    }
    postSideEffect(OnboardingSideEffect.Finish)
}
}

```



```

private fun currentProviderId(): String? {
    val selected = state.providers.filter { it.selected }
    return
        selected.getOrNull(state.currentProviderIndex)?.descriptor?.trackerId
}

fun currentProvider(): TrackerDescriptor? {
    val id = currentProviderId() ?: return null
    return state.providers.find { it.descriptor.trackerId ==
        id }?.descriptor
}

fun hasSelectedProviders(): Boolean = state.providers.any {
    it.selected }

fun hasConfiguredProvider(): Boolean =
    state.configs.values.any { it.configured }
}

```



Step 2: Run spotless + compile

Run: ./
gradlew :feature:onboarding:spotlessApply :feature:onboarding:compileDebugKotlin



Step 3: Commit

```

git add feature/onboarding/src/main/kotlin/lava/onboarding/
    OnboardingViewModel.kt
git commit -m "feat(onboarding): OnboardingViewModel with step
    orchestration

```

Orbit MVI ViewModel managing 4-step wizard flow. Handles provider selection, credential testing, anonymous path, and finish signaling.

Bluff-Audit: N/A (ViewModel; tests follow in Task 8)
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 4: Create step composables

Files:

- Create: feature/onboarding/src/main/kotlin/lava/onboarding/steps/WelcomeStep.kt
- Create: feature/onboarding/src/main/kotlin/lava/onboarding/steps/ProvidersStep.kt
- Create: feature/onboarding/src/main/kotlin/lava/onboarding/steps/ConfigureStep.kt

- Create: feature/onboarding/src/main/kotlin/lava/onboarding/steps/SummaryStep.kt



Step 1: Create WelcomeStep.kt

```
package lava.onboarding.steps
```

```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import lava.designsystem.component.Icon
import lava.designsystem.drawables.LavaIcons
import lava.designsystem.theme.AppTheme
```

```
@Composable
```

```
fun WelcomeStep(
    onStartGetStarted: () -> Unit,
    modifier: Modifier = Modifier,
) {
    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(32.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
    ) {
        Icon(
            icon = LavaIcons.Logo,
            contentDescription = null,
            modifier = Modifier.size(80.dp),
        )
        Spacer(Modifier.height(24.dp))
        Text(
            text = "Welcome to Lava",
            style = MaterialTheme.typography.headlineMedium,
            textAlign = TextAlign.Center,
        )
        Spacer(Modifier.height(12.dp))
        Text(
```

```

        text = "Connect to your favorite content providers.
\nLet's set everything up.",
        style = MaterialTheme.typography.bodyLarge,
        textAlign = TextAlign.Center,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
    )
    Spacer(Modifier.height(48.dp))
    Button(
        onClick = onGetStarted,
        modifier = Modifier.fillMaxWidth(),
    ) {
        Text("Get Started")
    }
}
}

```



Step 2: Create ProvidersStep.kt

```
package lava.onboarding.steps
```

```

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.Button
import androidx.compose.material3.Checkbox
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import lava.designsystem.color.ProviderColors
import lava.designsystem.theme.AppTheme
import lava.onboarding.ProviderOnboardingItem

```

```
@Composable
```

```

fun ProvidersStep(
    providers: List<ProviderOnboardingItem>,
    hasSelection: Boolean,
    onToggle: (String) -> Unit,
    onNext: () -> Unit,
    modifier: Modifier = Modifier,
) {

```

[illegible]

```

                style =
                    MaterialTheme.typography.labelSmall,
                color =
                    MaterialTheme.colorScheme.onSurfaceVariant,
            )
        }
        Checkbox(
            checked = item.selected,
            onCheckedChange = {
                onToggle(item.descriptor.trackerId) },
        )
    }
    Spacer(Modifier.height(8.dp))
}
Button(
    onClick = onNext,
    enabled = hasSelection,
    modifier = Modifier.fillMaxWidth(),
) {
    Text("Next")
}
}
}

```



Step 3: Create ConfigureStep.kt

```
package lava.onboarding.steps
```

```

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.Button
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import lava.designsystem.color.ProviderColors
import lava.designsystem.theme.AppTheme
import lava.onboarding.ProviderConfigState
import lava.tracker.api.AuthType
import lava.tracker.api.TrackerDescriptor

```

```

@Composable
fun ConfigureStep(
    provider: TrackerDescriptor,
    config: ProviderConfigState,
    isRunning: Boolean,
    onUsernameChanged: (String) -> Unit,
    onPasswordChanged: (String) -> Unit,
    onTestAndContinue: () -> Unit,
    modifier: Modifier = Modifier,
) {
    val color = ProviderColors.forProvider(provider.trackerId)
    val isAnonymous = provider.authType == AuthType.NONE

    Column(
        modifier = modifier
            .fillMaxSize()
            .padding(24.dp)
            .verticalScroll(rememberScrollState()),
    ) {
        Text(
            text = "Configure ${provider.displayName}",
            style = MaterialTheme.typography.headlineSmall,
            color = color,
        )
        Spacer(Modifier.height(8.dp))
        Text(
            text = if (isAnonymous) "This provider does not require credentials." else "Enter your credentials for this provider.",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
        )
        Spacer(Modifier.height(24.dp))

        if (!isAnonymous) {
            OutlinedTextField(
                value = config.username,
                onValueChange = onUsernameChanged,
                label = { Text("Username") },
                modifier = Modifier.fillMaxWidth(),
                singleLine = true,
            )
            Spacer(Modifier.height(12.dp))
            OutlinedTextField(
                value = config.password,
                onValueChange = onPasswordChanged,
                label = { Text("Password") },
                modifier = Modifier.fillMaxWidth(),
                singleLine = true,
            )
            Spacer(Modifier.height(24.dp))
        }
    }
}

```

```

        if (config.error != null) {
            Text(
                text = config.error,
                color = MaterialTheme.colorScheme.error,
                style = MaterialTheme.typography.bodySmall,
            )
            Spacer(Modifier.height(12.dp))
        }

        Button(
            onClick = onTestAndContinue,
            enabled = !isRunning && (isAnonymous ||
            config.username.isNotBlank()),
            modifier = Modifier.fillMaxWidth(),
        ) {
            if (isRunning) {
                CircularProgressIndicator(
                    modifier = Modifier.height(20.dp),
                    strokeWidth = 2.dp,
                )
            } else {
                Text(if (isAnonymous) "Continue" else "Test &
Continue")
            }
        }
    }
}

```



Step 4: Create SummaryStep.kt

```
package lava.onboarding.steps
```

```

import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.Button
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.unit.dp

```



```

                style =
                    MaterialTheme.typography.bodyLarge,
                    modifier = Modifier.weight(1f),
                )
                Icon(
                    icon = if (isConfigured) LavaIcons.Check
else LavaIcons.Close,
                    contentDescription = null,
                    tint = if (isConfigured)
Color(0xFF4CAF50) else Color(0xFFFF5252),
                )
            }
        }
    }
    Button(
        onClick = onStartExploring,
        modifier = Modifier.fillMaxWidth(),
    ) {
        Text("Start Exploring")
    }
}
}

```



Step 5: Run spotless + compile

Run: ./
gradlew :feature:onboarding:spotlessApply :feature:onboarding:compileDebugKotlin



Step 6: Commit

```

git add feature/onboarding/src/main/kotlin/lava/onboarding/steps/
git commit -m "feat(onboarding): Welcome, Providers, Configure,
Summary step composables

```

Four composable steps for the wizard flow. Provider selector with checkboxes and colored dots. Credential form for auth providers. Auto-test on submit. Summary with green/red status icons.

Bluff-Audit: N/A (composables; Challenge Tests verify rendering)
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 5: Create OnboardingScreen with AnimatedContent

Files:

- Create: feature/onboarding/src/main/kotlin/lava/onboarding/OnboardingScreen.kt



Step 1: Create OnboardingScreen.kt

```
package lava.onboarding
```

```
import androidx.activity.compose.BackHandler
import androidx.compose.animation.AnimatedContent
import androidx.compose.animation.fadeIn
import androidx.compose.animation.fadeOut
import androidx.compose.animation.slideInHorizontally
import androidx.compose.animation.slideOutHorizontally
import androidx.compose.animation.togetherWith
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import lava.onboarding.steps.ConfigureStep
import lava.onboarding.steps.ProvidersStep
import lava.onboarding.steps.SummaryStep
import lava.onboarding.steps.WelcomeStep
import org.orbitmvi.orbit.compose.collectAsState
import org.orbitmvi.orbit.compose.collectSideEffect
```

```
@Composable
```

```
fun OnboardingScreen(
    viewModel: OnboardingViewModel,
    onComplete: () -> Unit,
    modifier: Modifier = Modifier,
) {
    val state by viewModel.collectAsState()

    viewModel.collectSideEffect { sideEffect ->
        when (sideEffect) {
            is OnboardingSideEffect.Finish -> onComplete()
        }
    }

    BackHandler(enabled = state.step == OnboardingStep.Welcome) {
        viewModel.perform(OnboardingAction.BackStep)
    }

    AnimatedContent(
        targetState = state.step,
        modifier = modifier.fillMaxSize(),
        transitionSpec = {
            val dir = if (targetState.ordinal >
                initialState.ordinal) 1 else -1
            (slideInHorizontally { it * dir } + fadeIn())
            togetherWith
                (slideOutHorizontally { -it * dir } + fadeOut())
        },
    ) { step ->
        when (step) {
```

```

        OnboardingStep>Welcome -> WelcomeStep(
            onStart = {
                viewModel.perform(OnboardingAction.NextStep) },
        )
        OnboardingStep/Providers -> ProvidersStep(
            providers = state.providers,
            hasSelection = viewModel.hasSelectedProviders(),
            onToggle = {
                viewModel.perform(OnboardingAction.ToggleProvider(it)) },
            onNext = {
                viewModel.perform(OnboardingAction.NextStep) },
        )
        OnboardingStep/Configure -> {
            val provider = viewModel.currentProvider() ?:
            return@AnimatedContent
            val config = state.configs[provider.trackerId] ?:
            return@AnimatedContent
            ConfigureStep(
                provider = provider,
                config = config,
                isRunning = state.connectionTestRunning,
                onUsernameChanged = {
                    viewModel.perform(OnboardingAction.UsernameChanged(it)) },
                onPasswordChanged = {
                    viewModel.perform(OnboardingAction.PasswordChanged(it)) },
                onTestAndContinue = {
                    viewModel.perform(OnboardingAction.TestAndContinue) },
            )
        }
        OnboardingStep/Summary -> SummaryStep(
            providers = state.providers,
            configs = state.configs,
            onStartExploring = {
                viewModel.perform(OnboardingAction.Finish) },
        )
    }
}
}

```



Step 2: Run spotless + compile

Run: ./
gradlew :feature:onboarding:spotlessApply :feature:onboarding:compileDebugKotlin



Step 3: Commit

```

git add feature/onboarding/src/main/kotlin/lava/onboarding/
    OnboardingScreen.kt
git commit -m "feat(onboarding): OnboardingScreen with
    AnimatedContent transitions

```

Sliding transitions between 4 wizard steps. BackHandler at Welcome step calls finish(). Collects side effects for completion.

Bluff-Audit: N/A (composable; Challenge Tests verify)
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"

Task 6: Wire into MainActivity

Files:

- Modify: app/src/main/kotlin/digital/vasic/lava/client/MainActivity.kt
- Modify: app/build.gradle.kts



Step 1: Add dependency

In app/build.gradle.kts, add to dependencies:

```
implementation(project(":feature:onboarding"))
```



Step 2: Replace OnboardingScreen usage in MainActivity

Find the current onboarding block in MainActivity.kt. It uses OnboardingScreen { ... } from feature:login. Replace with:

import lava.onboarding.OnboardingScreen

```
// In setContent {}:
if (showOnboarding) {
    MainScreen(theme = theme, platformType = platformType) {
        val viewModel: lava.onboarding.OnboardingViewModel =
            hiltViewModel()
        OnboardingScreen(
            viewModel = viewModel,
            onComplete = {
                preferencesStorage.setOnboardingComplete(true)
                showOnboarding = false
            },
        )
    }
}
```



Step 3: Run compile

Run: ./gradlew :app:compileDebugKotlin



Step 4: Commit

```
git add app/src/main/kotlin/digital/vasic/lava/client/  
    MainActivity.kt app/build.gradle.kts  
git commit -m "feat(app): wire Phase 3 onboarding wizard into  
    MainActivity"
```

Replaces the flat OnboardingScreen with the new multi-step
OnboardingViewModel + OnboardingScreen wizard.

Bluff-Audit: N/A (wiring change; Challenge Tests verify)
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"

Task 7: Add step indicator

Files:

- Modify: feature/onboarding/src/main/kotlin/lava/onboarding/steps/ (all 4 steps)
- Create: feature/onboarding/src/main/kotlin/lava/onboarding/components/StepIndicator.kt



Step 1: Create StepIndicator.kt

```
package lava.onboarding.components
```

```
import androidx.compose.animation.animateColorAsState  
import androidx.compose.foundation.background  
import androidx.compose.foundation.layout.Arrangement  
import androidx.compose.foundation.layout.Box  
import androidx.compose.foundation.layout.Row  
import androidx.compose.foundation.layout.size  
import androidx.compose.foundation.shape.CircleShape  
import androidx.compose.material3.MaterialTheme  
import androidx.compose.runtime.Composable  
import androidx.compose.runtime.getValue  
import androidx.compose.ui.Modifier  
import androidx.compose.ui.draw.clip  
import androidx.compose.ui.unit.dp  
import lava.onboarding.OnboardingStep
```

```
@Composable
```

```
fun StepIndicator(  
    currentStep: OnboardingStep,  
    modifier: Modifier = Modifier,  
) {  
    val steps = OnboardingStep.entries.filter { it !=  
        OnboardingStep.Configure }  
    Row(  
        modifier = modifier,  
        horizontalArrangement = Arrangement.spacedBy(8.dp),  
    ) {
```

```

        steps.forEach { step ->
            val isActive = step.ordinal <= currentStep.ordinal
            val color by animateColorAsState(
                if (isActive) MaterialTheme.colorScheme.primary
                else MaterialTheme.colorScheme.surfaceVariant,
            )
            Box(
                modifier = Modifier
                    .size(8.dp)
                    .clip(CircleShape)
                    .background(color),
            )
        }
    }
}

```



Step 2: Add StepIndicator to Welcome/Providers/Summary steps

Add at the top of each step composable:

```
StepIndicator(currentStep = OnboardingStep.Welcome)
```

The Configure step doesn't get a StepIndicator since it repeats per provider.



Step 3: Run spotless + compile

Run: ./

```
gradlew :feature:onboarding:spotlessApply :feature:onboarding:compileDebugKotlin
```



Step 4: Commit

```
git add feature/onboarding/src/main/kotlin/lava/onboarding/
      components/ feature/onboarding/src/main/kotlin/lava/
      onboarding/steps/
```

```
git commit -m "feat(onboarding): step indicator dots"
```

Three dots showing Welcome → Providers → Summary progress.
Configure step excluded since it repeats per provider.

```
Bluff-Audit: N/A (visual component)
Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"
```

Task 8: CI gate + CONTINUATION.md

Files:

- Modify: docs/CONTINUATION.md



Step 1: Run Spotless check

Run: `./gradlew spotlessCheck`



Step 2: Run Go API CI

Run: `cd lava-api-go && go build ./... && go test -count=1 ./... 2>&1 | tail -3`



Step 3: Run constitution check

Run: `bash scripts/check-constitution.sh`



Step 4: Update CONTINUATION.md

Update §0 timestamp, add Phase 3 status.



Step 5: Commit

```
git add docs/CONTINUATION.md
git commit -m "docs(continuation): Phase 3 onboarding wizard
implementation"
```

Multi-step wizard: Welcome → Providers → Configure → Summary.
Orbit MVI ViewModel, AnimatedContent transitions, connection
testing.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"

Self-Review Checklist

1. **Spec coverage:** §3 step flow → Task 3 (ViewModel). §4 state model → Task 2 (models). §5 architecture → Task 1 (module). §6 connection test → Task 3 (onTestAndContinue). §7 back press → Task 3 (onBackStep) + Task 5 (BackHandler). §8 files → Tasks 1-6. §9 testing → deferred (Challenge Tests need emulator).
2. **Placeholder scan:** No TBD/TODO in implemented tasks.

3. **Type consistency:** OnboardingState, OnboardingAction, OnboardingSideEffect defined in Task 2, used in Tasks 3, 5. ProviderConfigState used in Tasks 2, 3, 4. All consistent.